

<!--

Copyright (c) 2026 Adam G. Sweeney agsweeney@gmail.com

SPDX-License-Identifier: MIT

Purpose: v1.0 draft specification for the Lightweight Ethernet Application Protocol.

-->

LEAP Protocol Specification

Status: Draft v1.0

Wire contract reference: `leap_protocol.h`

1. Scope

LEAP, the Lightweight Ethernet Application Protocol, is a vendor-neutral raw Ethernet automation protocol for deterministic device I/O, discovery, ownership, profile description, diagnostics, and safe behavior on communication loss.

LEAP operates without IP addressing. Controllers discover and manage devices by Ethernet MAC address and protocol-level identity data. LEAP is intended for private machine networks using standard Ethernet hardware and switches.

Target device classes include remote I/O modules, embedded controllers, motion controllers, sensors, and actuators.

LEAP defines a native protocol model for discovery, ownership, directory, process-data exchange, diagnostics, and profile-driven behavior. It does not use external fieldbus object models, register maps, or mapping layers as normative requirements.

2. Normative Language

The words `MUST`, `MUST NOT`, `SHALL`, `SHOULD`, and `MAY` define protocol requirements.

Items marked `REVIEW` remain draft decisions that need engineering approval before production use.

3. Design Principles

LEAP v1.0 is built around these principles:

1. No IP configuration is required.
2. Discovery and ownership are MAC-address based.
3. Standard Ethernet switches are supported.
4. Exactly one controller may own a device control lease at a time.
5. Cyclic process data is deterministic and explicitly sequenced.
6. Outputs enter a safe policy on owner loss, watchdog expiry, stale frames, or fault conditions.

7. Profiles are vendor-neutral where possible and vendor/project extensible where necessary.
8. Commissioning is human-friendly through mandatory identity and locate-device support.

4. Transport

LEAP frames are raw Ethernet frames on an isolated industrial cell network.

Item	Requirement
Physical medium	Standard Ethernet supported by the target hardware
Minimum MTU	1500 bytes
Minimum transmitted Ethernet payload	50 bytes, padded by the transport when needed
Maximum Ethernet payload v1	1500 bytes
LEAP header length v1	32 bytes
Maximum v1 payload after header	1468 bytes
Byte order	Little-endian for all multi-byte protocol fields, including raw Ethernet payload structures
Session addressing	Ethernet unicast
Discovery addressing	Ethernet broadcast and unicast as specified
Production EtherType	REVIEW, requires formal assignment
Development EtherType	0x88B6 default for local engineering and conformance testing (not a formally assigned LEAP EtherType)
Experimental alternate EtherType	0x88B5, configurable for private test networks (not a formally assigned LEAP EtherType)

Implementations MUST reject frames with malformed lengths, unsupported major versions, failed header checks, failed payload checks, invalid state for the requested service, unsupported service/message IDs, or payload sizes that exceed the receiving endpoint limits.

Cyclic process-data traffic MUST be unicast. Broadcast is permitted only for discovery and commissioning traffic that is explicitly defined as broadcast-safe.

The EtherType MUST be formally assigned before LEAP is used outside controlled local experiments. Implementations using experimental EtherTypes MUST make them configurable and MUST document those values as experimental.

4.1 Minimum Ethernet Frame Padding

LEAP transport drivers MUST ensure transmitted Ethernet frames are not runt frames. The transmitted Ethernet payload, measured after the Ethernet MAC header and before any FCS handled by the MAC, MUST be at least 50 bytes. Because the v1 LEAP header is 32 bytes, frames whose true LEAP `payload_length` is less than 18 bytes MUST be padded with trailing zero bytes by the transport.

The `payload_length` field MUST contain only the true LEAP payload length and MUST NOT include transport padding. `payload_crc32c` is calculated only over the true payload bytes. Receivers MUST ignore padding bytes beyond `header_length + payload_length`.

Receivers MUST validate LEAP frames against `header_length + payload_length`, not against a fixed padding size. Implementations MUST accept valid Ethernet payloads produced by standard IEEE 802.3 minimum-frame behavior and MUST NOT reject a frame solely because the observed physical payload padding differs from the sender's preferred 50 byte minimum transmit policy.

5. Layering

LEAP is split into a common frame layer and service namespaces:

Layer or Service	Service ID	Purpose
LEAP frame	n/a	Common Ethernet payload header, sequencing, flags, and integrity checks
LEAP-MGMT	0x0001	Sessions, owner lease, state control, watchdogs, and fault reset
LEAP-DISC	0x0002	Discovery, identity bootstrap, and locate-device commissioning
LEAP-DIR	0x0003	Directory, object access, profile metadata, and endpoint table
LEAP-PD	0x0010	Cyclic process-data read, write, and combined exchange
LEAP-DIAG	0x0020	Counters, timing, events, faults, and trace marks
Vendor extension	0x8000..0xFFFE	Private extensions
Reserved invalid	0xFFFF	Invalid service ID

Service IDs below 0x8000 are protocol-governed. New standard services require an update to this specification and `leap_protocol.h`.

Transport code owns frame validation, service routing, sessions, endpoint buffers, state, and diagnostics. Device application code owns endpoint application, input sampling, and profile-specific status population.

6. Common Frame Header

Every Ethernet payload begins with `LeapHeader`, a fixed 32-byte header.

Offset	Size	Field	Description
0	4	magic	v1 wire marker bytes <code>LEAP</code> ; <code>LEAP_MAGIC_U32 = 0x5041454C</code>
4	1	version_major	Major protocol version; v1 uses <code>1</code>
5	1	version_minor	Minor protocol version; v1.0 uses <code>0</code>
6	1	header_length	Header bytes; v1 uses <code>32</code>
7	1	flags	Frame flags
8	2	service_id	Service namespace
10	2	message_type	Service-local message type
12	4	session_id	<code>0</code> for discovery and pre-session traffic
16	4	sequence	Sender frame sequence number
20	4	ack_sequence	Latest peer sequence observed, or <code>0</code>
24	2	payload_length	Payload bytes after this header
26	2	header_crc16	CRC-16 over the header with this field zeroed
28	4	payload_crc32c	CRC-32C over payload, or <code>0</code> when explicitly disabled

All integer fields in `LeapHeader` and all LEAP payload structures are little-endian on the wire. This rule applies even though Ethernet protocol fields are commonly documented in network byte order. Packet analyzers and conformance tests MUST decode LEAP protocol fields as little-endian.

6.1 Integrity Algorithms

LEAP v1.0 integrity algorithms are fixed as follows:

Header CRC (`header_crc16`):

- algorithm: `CRC-16/XMODEM` (also known as CRC-16/ACORN)
- polynomial: `0x1021`
- initial value: `0x0000`
- input reflected: `false`
- output reflected: `false`
- xor-out: `0x0000`
- check value for ASCII `123456789`: `0x31c3`
- coverage: all `header_length` bytes with `header_crc16` field bytes set to zero

Payload CRC (`payload_crc32c`):

- algorithm: `CRC-32C` (Castagnoli)
- polynomial: `0x1EDC6F41` (normal) / `0x82F63B78` (reflected)
- initial value: `0xFFFFFFFF`
- input reflected: `true`
- output reflected: `true`
- xor-out: `0xFFFFFFFF`
- check value for ASCII `123456789`: `0xE3069283`
- coverage: true payload bytes only (`payload_length`), excluding any transport padding

Golden frame vector (valid CRCs):

- payload bytes: ASCII `LEAP` (`4C 45 41 50`)
- payload CRC-32C: `0x2FCD7428`
- header bytes with `header_crc16 = 0x0000` :
`4c454150010020010200010000000000010000000000000040000002874cd2f`
- resulting header CRC-16/XMODEM: `0xE33C`

Conformance vectors MUST include at least one valid frame and at least one bad-header-CRC and bad-payload-CRC rejection vector.

6.2 Header Flags

Bit	Mask	Name	Meaning
0	0x01	ACK_REQUESTED	Receiver SHOULD send an explicit response
1	0x02	RESPONSE	Frame is a response to a previous request
2	0x04	ERROR	Response payload begins with <code>LeapErrorPayload</code>
3	0x08	BROADCAST	Sender intentionally used broadcast destination
4	0x10	NO_PAYLOAD_CRC	Payload CRC field is ignored; only allowed where specified
5	0x20	FRAGMENTED	Payload begins with <code>LeapFragmentHeader</code>
6	0x40	TIME_VALID	Time fields in the payload are meaningful
7	0x80	RESERVED7	Send as 0 ; receivers ignore unless a future version assigns it

Reserved flags MUST be sent as 0 . Receivers SHOULD ignore unknown reserved flags only when the major version is supported and the frame can still be validated.

7. Common Payload Types

7.1 TLV Encoding

Metadata messages MAY contain type-length-value fields.

Offset	Size	Field	Description
0	2	type	Registry-defined TLV type
2	2	length	Value bytes
4	length	value	Field payload

TLV values are padded to a 4-byte boundary. Receivers MUST skip unknown TLV types with valid lengths and MUST reject TLVs whose declared length exceeds the remaining payload.

Serialization and parsing code MUST advance by the 4-byte padded TLV length, not by the unpadded value length. Implementations MUST NOT perform unaligned native loads from TLV value pointers; parsers should copy or decode little-endian integer fields byte-wise when alignment is not guaranteed by the host CPU.

7.2 TLV Registry

Type	Name	Value
0x0001	PROTOCOL_VERSION_RANGE	LeapVersionRange
0x0002	DEVICE_NAME	UTF-8 or ASCII device name bytes
0x0003	VENDOR_ID	Vendor identifier
0x0004	PRODUCT_CODE	Product code
0x0005	REVISION	Product or device revision
0x0006	SERIAL_NUMBER	Serial number
0x0007	PRIMARY_MAC	Primary MAC address
0x0008	SUPPORTED_SERVICES	Packed list of 16-bit service IDs
0x0009	DEFAULT_PROFILE_ID	Default profile ID
0x000A	ACTIVE_PROFILE_ID	Currently active profile ID
0x000B	CURRENT_STATE	Current LeapState
0x000C	ACTIVE_OWNER_MAC	Active owner MAC, when any
0x000D	DEVICE_IDENTITY	LeapIdentity

Type	Name	Value
0x000E	ENDPOINT_DESCRIPTOR	LeapEndpointDescriptor
0x000F	PROFILE_DESCRIPTOR	LeapProfileDescriptor
0x0010	LOCATE_CAPABILITY	Locate-device capability flags
0x0011	SWITCH_SAFE_CAPABILITY	Switch-safe behavior capabilities
0x0012	MAX_FRAME_AGE_US	Maximum accepted cyclic frame age
0x0013	PROFILE_NAME	Profile name bytes
0x0014	SCHEMA_OBJECT_ID	Object ID containing schema data
0x8000..	Vendor TLVs	Vendor/project extension

7.3 Error Payload

All error responses set the `ERROR` and `RESPONSE` flags and begin with `LeapErrorPayload`.

Offset	Size	Field	Description
0	2	status_code	LEAP status code
2	2	detail_code	Service-local detail code
4	4	rejected_sequence	Sequence number that failed
8	4	affected_offset	Optional byte or object offset
12	4	affected_length	Optional byte or object length

7.4 Status Codes

Code	Name	Meaning
0x0000	OK	Request accepted
0x0001	UNSUPPORTED_VERSION	Major version is not supported
0x0002	BAD_LENGTH	Frame, header, TLV, or payload length invalid
0x0003	BAD_CHECK	Header or payload integrity check failed
0x0004	UNSUPPORTED_SERVICE	Service is unknown or disabled
0x0005	UNSUPPORTED_MESSAGE	Message is unknown for the service
0x0006	INVALID_STATE	Message is not valid in current state

Code	Name	Meaning
0x0007	NOT_OWNER	Sender does not own the active control lease
0x0008	LEASE_EXPIRED	Active lease expired before request arrived
0x0009	BUSY	Device cannot complete the request now
0x000A	FAULTED	Device is faulted and requires reset or safe action
0x000B	RANGE	Object, endpoint, offset, or length out of range
0x000C	RATE_LIMITED	Sender exceeded configured service rate
0x000D	STALE_FRAME	Cyclic frame is older than the accepted age
0x000E	OUT_OF_ORDER	Sequence ordering violates service rules
0x000F	DUPLICATE_SEQUENCE	Duplicate frame or process sequence
0x0010	PROFILE_MISMATCH	Requested profile does not match active profile
0x0011	WATCHDOG_EXPIRED	Process watchdog expired
0x0012	LOCATE_UNAVAILABLE	Locate-device behavior is unavailable
0x0013	AUTH_REQUIRED	Authentication is required by an extension
0x0014	PERMISSION_DENIED	Authenticated or configured role lacks permission
0x8000..	Vendor status	Vendor/project extension

8. Device Identity

Every conformant LEAP device SHALL expose a mandatory identity object containing:

- primary MAC address
- vendor ID
- product code
- serial number
- hardware revision
- firmware revision
- device capability flags
- human-readable device name, when available

The fixed identity object is `LeapIdentity`.

Offset	Size	Field	Description
0	6	primary_mac	Primary Ethernet MAC address

Offset	Size	Field	Description
6	2	vendor_id	Vendor identifier
8	4	product_code	Product code
12	4	serial_number	Device serial number
16	2	hardware_revision	Hardware revision
18	2	firmware_revision	Firmware revision
20	4	device_capability_flags	Device capability flags

Strings and optional identity fields SHOULD be exposed as TLVs or directory objects, not embedded in fixed wire structures.

9. Discovery and Commissioning Service

LEAP-DISC uses service ID 0x0002. Discovery uses session_id = 0.

Discovery MUST NOT change outputs, owner leases, process state, or retained configuration.

Message Type	Name	Direction	Purpose
0x0001	HELLO	Controller broadcast to devices	Discover devices
0x0002	HELLO_REPLY	Device unicast to controller	Return basic identity and state
0x0003	IDENTIFY	Controller request	Request identity from one device
0x0004	IDENTIFY_REPLY	Device response	Return identity and capabilities
0x0005	LOCATE_DEVICE	Controller request	Physically identify a device
0x0006	LOCATE_DEVICE_REPLY	Device response	Report locate-device status

9.1 Discovery Payloads

LeapHelloRequest contains controller capability flags and reserved bytes.

LeapHelloReply and LeapIdentifyReply contain fixed identity, default and active profile IDs, current state, supported services, active owner MAC, and locate-device capability flags.

Discovery replies SHOULD also expose TLVs for protocol version range, device name, supported services, profile descriptors, endpoint descriptors, and switch-safe capability.

9.2 Supported Service List

Discovery replies MUST NOT encode supported services as a bitmap because LEAP service IDs are sparse (0x0001, 0x0002, 0x0003, 0x0010, 0x0020, and vendor ranges). `LeapHelloReply.supported_service_count` and `LeapIdentifyReply.supported_service_count` contain the number of 16-bit service IDs that immediately follow the fixed reply structure. Each entry is a little-endian `service_id` from the service registry.

If the `SUPPORTED_SERVICES` TLV is present, its value uses the same packed little-endian `uint16_t service_id[]` encoding. Controllers MUST ignore duplicate service IDs and MUST reject malformed service lists whose byte length is not a multiple of two.

9.3 Locate Device

All conformant LEAP v1.0 devices MUST support `LOCATE_DEVICE` and `LOCATE_DEVICE_REPLY`.

`LOCATE_DEVICE` SHALL:

- activate a visual indicator when available
- activate a buzzer when present and requested
- use the requested duration and pattern when supported
- allow cancellation through the cancel flag
- never affect outputs
- never affect ownership
- never alter retained configuration

Locate-device patterns:

Value	Name
0	DEFAULT
1	SLOW_BLINK
2	FAST_BLINK
3	DOUBLE_BLINK
4	SOLID
255	CUSTOM

Locate-device flags:

Mask	Name
0x01	LED

	Mask	Name
	0x02	BUZZER
	0x04	DISPLAY
	0x80	CANCEL

10. Management Service

LEAP-MGMT uses service ID 0x0001. It controls sessions, ownership, device state, watchdog setup, heartbeat, fault reset, and owner release.

Message Type	Name	Purpose
0x0001	OPEN_SESSION	Request a session and optional owner lease
0x0002	OPEN_SESSION_REPLY	Assign session ID and lease parameters
0x0003	CLOSE_SESSION	Release a session and optional owner lease
0x0004	HEARTBEAT	Refresh owner lease and report sequence health
0x0005	SET_STATE	Request device state transition
0x0006	STATE_REPLY	Report accepted and current state
0x0007	FAULT_RESET	Clear eligible protocol or application faults
0x0008	OWNER_RELEASE	Place outputs in configured safe state and release ownership

10.1 Session and Ownership Flags

Open-session request flags:

Mask	Name	Meaning
0x0001	REQUEST_OWNER	Request the active owner lease
0x0002	OBSERVER_ONLY	Request observation only
0x0004	STEAL_EXPIRED	Claim ownership if the previous lease is expired
0x0008	REBOOT_RECOVERY	Same-MAC controller recovery from a stale pre-reboot owner session

Session reply flags:

Mask	Name	Meaning
0x0001	OWNER	Session owns the control lease

Mask	Name	Meaning
0x0002	OBSERVER	Session may observe but not write outputs
0x0004	LEASE_ACTIVE	Owner lease is currently active

10.2 Owner Lease

An owner lease is required before LEAP-PD can apply output data in OP.

- Lease and process watchdog durations are negotiated during OPEN_SESSION.
- A controller MUST refresh the lease with HEARTBEAT or accepted LEAP-PD traffic before the deadline.
- A device MUST transition to SAFE and apply configured safe outputs when the owner lease expires unless the active profile requires a stricter FAULT.
- A second controller MAY observe state, directory data, and diagnostics, but MUST NOT write outputs until the owner releases or times out.
- Output writes from non-owner sessions MUST be rejected with NOT_OWNER.

10.3 Session Lifecycle

Session lifecycle behavior is fixed for v1.0:

- assigned_session_id MUST be non-zero.
- Devices MUST allocate session IDs from a monotonically increasing 32-bit counter and skip 0.
- On counter wrap, session ID allocation MUST roll from 0xFFFFFFFF to 0x00000001.
- Device reboot invalidates all previous session IDs and owner/observer state.
- Frames received with an unknown or invalidated session_id MUST be rejected with INVALID_STATE or LEASE_EXPIRED.
- Observer sessions MUST expire after inactivity. The default observer timeout is the granted lease time returned by OPEN_SESSION_REPLY unless overridden by profile or management configuration.
- After controller reboot or link-loss recovery, controllers MUST rediscover, reopen a session, reread directory/profile state, and re-arm state transitions before requesting OP.

Reboot owner recovery:

- If OPEN_SESSION includes REBOOT_RECOVERY, the device is currently owned, and the incoming frame source MAC exactly matches the active owner MAC, the device MAY immediately terminate the previous owner session before lease expiry.
- On accepted recovery, the device MUST invalidate the old session ID, stop applying process outputs, transition OP -> CONFIGURED, and require profile confirmation/selection before returning to SAFE or OP.

- If the incoming frame source MAC does not match the active owner MAC, the device MUST reject the request with `BUSY`, `NOT_OWNER`, or `PERMISSION_DENIED`.

10.3.1 Controller-Side Session Continuity

Controllers MUST monitor per-device session continuity across all owned devices.

A controller MUST treat any of the following as a session-reset event for a previously known device MAC:

- Receipt of a `HELLO_REPLY` or `IDENTIFY_REPLY` from that MAC with the device reporting `BOOT` or `INIT` state while the controller holds an active `session_id` for that device.
- Receipt of any LEAP frame from that MAC with `session_id = 0` while the controller holds a non-zero active `session_id`.
- Receipt of any LEAP frame from that MAC carrying a `session_id` that does not match the controller's recorded active session for that device.

On detecting a session-reset event, the controller MUST:

1. Immediately invalidate its locally held `session_id` and lease state for that device.
2. Halt all cyclic `LEAP-PD` output writes to that device.
3. Re-execute the full discovery, session open, directory read, profile confirmation, and state-progression sequence before resuming output writes.

Controllers MUST NOT rely solely on device-side rejection of stale `session_id` frames as the safety gate for this condition. Sending cyclic output writes using a stale `session_id` to a freshly rebooted device is a controller-side protocol error regardless of whether the device rejects those frames.

10.4 Device State Machine

State	Value	Meaning	Output Behavior
<code>BOOT</code>	<code>0x0000</code>	Firmware has not completed transport init	Application defaults
<code>INIT</code>	<code>0x0001</code>	Protocol ready, no active process profile	Safe outputs
<code>CONFIGURED</code>	<code>0x0002</code>	Directory and endpoint profile selected	Safe outputs
<code>SAFE</code>	<code>0x0003</code>	Owner may exchange data, outputs not applied or forced safe	Safe outputs
<code>OP</code>	<code>0x0004</code>	Cyclic process data is active	Owned outputs applied
<code>FAULT</code>	<code>0x0005</code>	Protocol or application fault active	Fault-policy outputs

Required transitions:

- `BOOT -> INIT` after transport initialization.
- `INIT -> CONFIGURED` after a valid profile is selected or defaulted.
- `CONFIGURED -> SAFE` after an owner lease is granted.
- `SAFE -> OP` only when endpoints validate and watchdogs are armed.
- `OP -> SAFE` on owner release, heartbeat loss, communication loss, stale process data, or explicit safe request.
- Any state `-> FAULT` on unrecoverable protocol or application fault.
- `FAULT -> INIT` only through accepted `FAULT_RESET` and application approval.

Devices MUST reject unsupported transitions with `INVALID_STATE`.

Devices with a single mandatory default profile MAY enter `CONFIGURED` automatically after initialization. They still MUST report the active profile and endpoint table through `LEAP-DIR` before entering `OP`.

10.5 LEAP-MGMT Session Transition Summary

Current condition	Incoming management action	Required result
no active session	<code>OPEN_SESSION</code>	allocate non-zero session ID; set owner/observer flags; remain in <code>INIT</code> or <code>CONFIGURED</code> until state request
owner active, different source MAC	<code>OPEN_SESSION</code> owner request	reject with <code>BUSY</code> or <code>NOT_OWNER</code>
owner active, same source MAC, <code>REBOOT_RECOVERY</code> set	<code>OPEN_SESSION</code>	invalidate old owner session; force <code>OP -> CONFIGURED</code> ; open new session
owner active, lease expired	<code>OPEN_SESSION</code> with <code>STEAL_EXPIRED</code>	assign new owner session and keep outputs safe until explicit state progression
observer session inactive past timeout	any frame from observer session	reject as invalidated session
valid owner session	<code>CLOSE_SESSION</code> or <code>OWNER_RELEASE</code>	release owner lease and transition to <code>SAFE</code>
any active session	device reboot	invalidate all sessions; clear owner/observer; transition through boot/init path

11. Directory Service

`LEAP-DIR` uses service ID `0x0003`. It provides a live object directory, profile metadata, and endpoint descriptors as the normative source of runtime configuration and capability data.

Message Type	Name	Purpose
0x0001	READ_DIRECTORY	Read identity, services, profiles, endpoints, and TLVs
0x0002	READ_DIRECTORY_REPLY	Return TLV list or fixed object response
0x0003	READ_OBJECT	Read one object by ID and offset
0x0004	READ_OBJECT_REPLY	Return object bytes and metadata
0x0005	WRITE_OBJECT	Write configurable object bytes
0x0006	WRITE_OBJECT_REPLY	Return accepted write range
0x0007	SELECT_PROFILE	Choose a process-data profile
0x0008	PROFILE_REPLY	Return active profile and endpoint table

11.1 Object ID Space

Object IDs are 32-bit values. The high 16 bits are the namespace and the low 16 bits are the object number.

Namespace	Name	Meaning
0x0001	IDENTITY	Identity and protocol information
0x0002	MGMT_CONFIG	Management configuration
0x0003	ENDPOINT_PROFILE	Endpoint and profile metadata
0x0004	PERSISTENT_CONFIG	Persistent device configuration
0x0005	DIAGNOSTICS	Diagnostics and counters
0x8000..0xFFFE	Vendor/project extension	Private object space

11.2 Endpoint Descriptor

Each cyclic data window is described by `LeapEndpointDescriptor`.

Offset	Size	Field	Description
0	2	endpoint_id	Stable endpoint identifier
2	1	direction	1=controller_to_device, 2=device_to_controller
3	1	flags	Endpoint flags
4	4	profile_id	Profile that owns this endpoint
8	2	byte_length	Endpoint payload size

Offset	Size	Field	Description
10	1	alignment	Required byte alignment
11	1	reserved	Send 0
12	4	schema_object_id	Object describing field layout

Endpoint flags:

Mask	Name
0x01	FIXED
0x02	OPTIONAL
0x04	RETAINED
0x08	SAFE_STATE
0x10	FAULT_STATE
0x20	READABLE_SAFE
0x40	READABLE_FAULT
0x80	RESERVED7

Standard endpoint IDs:

Endpoint ID	Name
0x0001	COMMAND
0x0002	STATUS
0x0010	DIGITAL_OUTPUTS
0x0011	DIGITAL_INPUTS
0x0020	ANALOG_OUTPUTS
0x0021	ANALOG_INPUTS
0x0030	SAFE_OUTPUTS
0x0031	FAULT_OUTPUTS
0x8000..	Vendor endpoint

12. Profile Registry

Profile IDs are 32-bit values. Once published, a profile ID is immutable. Incompatible changes to endpoint layout, byte order, field meaning, or required semantics require a new profile ID.

Initial standard profile IDs:

Profile ID	Name
0x00000000	NONE
0x00010001	DIGITAL_IO_8X8
0x00010002	DIGITAL_IO_16X16
0x00010003	DIGITAL_IO_32X32
0x00020001	ANALOG_IO_4AI
0x00020002	ANALOG_IO_4AO
0x00020003	ANALOG_IO_4AI_4AO
0x00030001	MIXED_IO_16DI_16DO_4AI_4AO
0x00040001	MOTION_SINGLE_AXIS
0x00040002	MOTION_MULTI_AXIS
0x80000000..	Vendor/project profiles

Profiles are specified independently from the base protocol. This document defines the registry, endpoint model, and conformance requirements that profile specifications must follow.

Motion behavior is intentionally not solved in the base transport. Distributed clocking, synchronized sampling, interpolation periods, and deterministic multi-axis coordination **MUST** be defined by motion profile specifications or by a future time-synchronization service. The base protocol only provides ownership, state, directory, diagnostics, and process-data exchange primitives.

`MOTION_SINGLE_AXIS` and `MOTION_MULTI_AXIS` profile IDs reserve namespace only. They **MUST NOT** be treated as interoperable motion profiles until a dedicated motion profile specification is published that normatively defines all of the following:

- distributed clock synchronization mechanism, accuracy requirements, and maximum permissible clock drift
- interpolation period, trajectory representation, and ramp profile rules
- axis enable and disable semantics and required sequencing
- homing procedures and reference position establishment

- following error detection, configurable threshold, and fault escalation policy
- hardware travel limits and software travel limit enforcement
- torque limiting and velocity limiting
- drive-disable output behavior on fault, safe-state entry, and communication loss
- axis status word layout, mode-of-operation flags, and error code mapping
- coordinated multi-axis grouping and synchronization rules where applicable

Implementations MUST NOT claim motion interoperability with other LEAP devices on the basis of `MOTION_SINGLE_AXIS` or `MOTION_MULTI_AXIS` profile ID alone until all of the above are normatively specified in a published LEAP motion profile document. Products shipping with these profile IDs before that document exists MUST document them as vendor-private profiles and MUST NOT represent them as conforming to a LEAP standard motion profile.

12.1 Golden Reference Profile: DIGITAL_IO_16X16

`DIGITAL_IO_16X16` (profile ID `0x00010002`) is the v1.0 golden reference profile for basic interoperability.

Reference process-data image (`8` bytes, packed):

Byte Offset	Size	Field	Description
<code>0</code>	<code>2</code>	<code>digital_inputs</code>	bits <code>0..15</code> map to digital input channels <code>1..16</code>
<code>2</code>	<code>2</code>	<code>digital_outputs</code>	bits <code>0..15</code> map to digital output channels <code>1..16</code>
<code>4</code>	<code>2</code>	<code>io_status</code>	status bitfield for module diagnostics
<code>6</code>	<code>1</code>	<code>v_field_supply</code>	field supply voltage in <code>0.1 v</code> units
<code>7</code>	<code>1</code>	<code>reserved0</code>	reserved, send <code>0</code>

Reference `io_status` bits:

Mask	Name	Meaning
<code>0x0001</code>	<code>FIELD_POWER_FAULT</code>	external field I/O power missing or low
<code>0x0002</code>	<code>OUTPUT_SHORT</code>	overcurrent or short circuit detected
<code>0x0004</code>	<code>WIRE_BREAK</code>	open circuit detected on supervised channels
<code>0x0008</code>	<code>THERMAL_WARN</code>	output driver overtemperature warning

When this profile is used with small LEAP-PD payloads, transport padding still follows the minimum Ethernet payload rule. `payload_length` remains the true data length and padding bytes are excluded from `payload_crc32c`.

13. Process Data Service

`LEAP-PD` uses service ID `0x0010`. It is the cyclic data path. `LEAP-PD` is valid only for the session owner after the device reaches `SAFE`; output application is valid only in `OP`.

Message Type	Name	Purpose
<code>0x0001</code>	<code>WRITE_ENDPOINT</code>	Write one controller-to-device endpoint
<code>0x0002</code>	<code>READ_ENDPOINT</code>	Request one device-to-controller endpoint
<code>0x0003</code>	<code>ENDPOINT_DATA</code>	Response containing endpoint data
<code>0x0004</code>	<code>EXCHANGE_ENDPOINTS</code>	Write command endpoint and read status endpoint
<code>0x0005</code>	<code>EXCHANGE_REPLY</code>	Combined exchange response

13.1 Process-Data Flags

Mask	Name	Meaning
<code>0x0001</code>	<code>APPLY_OUTPUTS</code>	Request output application when state permits
<code>0x0002</code>	<code>READBACK_REQUIRED</code>	Controller requires readback data
<code>0x0004</code>	<code>TIMESTAMP_VALID</code>	Timestamp fields are valid
<code>0x0008</code>	<code>ALLOW_SKIP</code>	Profile may accept skipped process sequence values

13.2 Single-Endpoint Access

`WRITE_ENDPOINT`, `READ_ENDPOINT`, and `ENDPOINT_DATA` use `LeapEndpointDataHeader`.

Offset	Size	Field	Description
<code>0</code>	<code>2</code>	<code>endpoint_id</code>	Endpoint being accessed
<code>2</code>	<code>2</code>	<code>endpoint_offset</code>	Byte offset within endpoint
<code>4</code>	<code>2</code>	<code>data_length</code>	Bytes in this operation
<code>6</code>	<code>2</code>	<code>endpoint_flags</code>	Process-data flags
<code>8</code>	<code>4</code>	<code>process_sequence</code>	Application-level process sequence
<code>12</code>	<code>4</code>	<code>cycle_time_us</code>	Requested or measured cycle period
<code>16</code>	<code>8</code>	<code>controller_timestamp_us</code>	Controller timestamp
<code>24</code>	<code>4</code>	<code>max_frame_age_us</code>	Maximum acceptable frame age

Offset	Size	Field	Description
28	4	profile_id	Expected active profile ID
32	N	data	Endpoint data when applicable

13.3 Combined Exchange

`EXCHANGE_ENDPOINTS` writes one controller-to-device endpoint and reads one device-to-controller endpoint in a single transaction. The request and reply begin with `LeapExchangeHeader`.

Offset	Size	Field	Description
0	2	write_endpoint_id	Command/output endpoint
2	2	read_endpoint_id	Status/input endpoint
4	2	write_length	Command/output bytes following header
6	2	read_length	Requested status/input bytes
8	4	process_sequence	Command sequence
12	4	cycle_time_us	Requested cycle period, or 0
16	8	controller_timestamp_us	Controller timestamp
24	4	max_frame_age_us	Maximum accepted age for this frame
28	4	profile_id	Expected active profile ID
32	2	exchange_flags	Process-data flags
34	2	reserved	Send 0
36	write_length	write_data	Command/output image
36 + write_length	read_length	read_reservation	Zero-filled by sender in request

The response preserves the same layout and writes status/input bytes into the `read_reservation` range. Output bytes always come before input bytes in the combined payload.

`LeapExchangeStatus` MAY be returned by profiles or diagnostic exchanges where a separate status block is required.

13.4 Switch-Safe Operation

LEAP v1.0 SHALL support operation through standard Ethernet switches.

Rules:

- Controllers SHALL use one outstanding cyclic exchange per device/session owner pair. A controller MUST NOT pipeline multiple cyclic exchanges to the same owned device unless a future profile or protocol revision explicitly allows it.
- Cyclic traffic MUST be unicast.
- Discovery broadcast is permitted.
- Devices MUST reject stale process frames older than profile or request limits when a shared time basis or bounded age estimate is available.
- Devices MUST reject out-of-order process frames unless the active profile explicitly allows a skip.
- Stale, late, duplicate, and out-of-order frames MUST NOT apply outputs.
- Late and stale frames SHOULD be counted in diagnostics.
- Outputs are applied only in `OP`, only from the owner session, and only after frame, profile, endpoint, length, sequence, and age validation pass.

`controller_timestamp_us` and `max_frame_age_us` provide stale-frame protection only when the device can compare controller time to a shared time basis or can derive a bounded age estimate from local receive time. If neither condition is true, the device MUST NOT claim timestamp-based stale-frame enforcement. It MUST still enforce local receive-time watchdogs, process sequence ordering, duplicate detection, and one-outstanding-exchange behavior. Devices SHOULD report timestamp or age-enforcement limitations through directory capabilities or diagnostics.

Deployments MAY carry LEAP frames inside IEEE 802.1Q VLAN-tagged Ethernet frames when the network supports it. VLAN priority configuration is outside the v1 base wire format, but managed-switch deployments SHOULD document any Class of Service or traffic-class settings used for cyclic `LEAP-PD` traffic.

Controllers and devices SHOULD use `sequence`, `ack_sequence`, and timing diagnostics to detect rising latency before watchdog expiry. Implementations MAY increment `SWITCH_CONGESTION_HINTS` when latency spikes or jitter exceed profile-defined warning thresholds without a hard frame loss.

13.4.1 Stale-Frame Category A: Local-Time-Only Devices

Category A applies when the device has no shared synchronized network time with the controller.

Required behavior:

- The device MUST track `highest_process_sequence` for each active owner session. See §13.6 for the full acceptance table, status codes, and reset rules.
- If incoming `process_sequence <= highest_process_sequence`, the frame is stale

or out-of-order and MUST be dropped without output updates.

- If incoming `process_sequence > highest_process_sequence`, the frame is sequence-valid and `highest_process_sequence` is updated.
- The device MUST monitor local inter-arrival timing for valid cyclic frames. If inter-arrival exceeds $1.5 * \text{expected_cycle_time_us}$ (where `expected_cycle_time_us` is profile-defined or negotiated), the device SHOULD increment `SWITCH_CONGESTION_HINTS`.
- Category A devices continue operation until watchdog or lease rules force a transition; congestion warnings do not by themselves authorize output changes.

Consecutive-miss thresholds for Category A devices:

- **1 or 2 consecutive missed cycles:** The device MUST increment `PROCESS_CYCLES_MISSED` and SHOULD increment `SWITCH_CONGESTION_HINTS`. Output application continues from the last valid frame subject to all other watchdog and lease rules. A device MUST NOT transition state on a single missed cycle attributable to ordinary switch queuing jitter.
- **3 or more consecutive missed cycles:** The device MUST cease output application and transition from `OP` to `SAFE`. This threshold is the normative default; profiles MAY lower it but MUST NOT raise it above `3`.

A missed cycle is counted when local inter-arrival time since the last sequence-valid frame exceeds `cycle_time_us` from that frame. When `cycle_time_us` is zero or unavailable, the device MUST use the process-data timeout negotiated at `OPEN_SESSION` as the cycle-boundary reference. The missed-cycle counter MUST reset to zero on every accepted sequence-valid frame.

13.4.2 Stale-Frame Category B: Shared-Time Devices

Category B applies when device and controller share a synchronized time basis.

LEAP v1.0 does not define or mandate a specific time synchronization mechanism. A "shared time basis" means any deployment-provided arrangement under which both the device and its owner controller maintain a common microsecond-resolution clock that advances at the same rate. Acceptable sources include IEEE 1588 Precision Time Protocol (PTP), a vendor-defined master timestamp broadcast, or any other mechanism that bounds clock offset to within a small fraction of the minimum `max_frame_age_us` value in use. Implementations MUST document the synchronization source used and MUST report it through directory capabilities or the `SWITCH_SAFE_CAPABILITY` TLV.

If a device loses its time synchronization source at runtime, it MUST immediately revert to Category A behavior and MUST NOT continue applying Category B age checks until synchronization is re-established and verified.

Required behavior:

- The device computes frame age from `controller_timestamp_us` and local arrival time using the shared time basis.
- If computed frame age exceeds `max_frame_age_us`, the frame is stale and MUST

be dropped without output updates.

- The device SHOULD increment late-frame counters when stale frames are dropped by age checks.
- If consecutive stale or missing frames span longer than profile watchdog/lease limits, the state machine MUST transition to `SAFE` (or `FAULT` if profile policy requires).

Timestamp arithmetic and jitter handling for Category B devices:

Frame age MUST be computed using unsigned wrapping subtraction on the 64-bit `controller_timestamp_us` field:

```
frame_age_us = (local_arrival_time_us - controller_timestamp_us)
```

Both operands are treated as unsigned 64-bit values. Signed arithmetic MUST NOT be used; it produces incorrect results whenever the controller timestamp has wrapped past the 2^{64} microsecond boundary (approximately 584,542 years from epoch, but reachable when timestamps are relative to device boot or power cycle).

Implementations MAY apply a normative jitter margin when the deployment includes managed switches with bounded and measurable queuing delays:

```
frame_age_us <= max_frame_age_us + jitter_margin_us
```

`jitter_margin_us` MUST be profile-defined and MUST NOT exceed one full expected cycle period (`cycle_time_us`). When no profile-specific jitter margin is defined, implementations MUST use zero and rely solely on `max_frame_age_us`. Jitter margins MUST NOT be applied to compensate for clock synchronization drift; that drift must be bounded by the time-synchronization subsystem itself.

Category A and Category B are capability modes. Devices MUST report which mode is implemented through discovery or directory capability data.

13.5 Reliability Rules

- The frame `sequence` detects duplicate Ethernet frames.
- The `process_sequence` detects repeated, regressive, or skipped application cycles. Normative device enforcement rules are defined in §13.6.
- A device MAY drop duplicate `LEAP-PD` frames after re-sending the previous response.
- Cyclic `LEAP-PD` traffic SHOULD NOT block waiting for retransmission; the next cycle is the recovery path.
- Configuration and management traffic SHOULD use explicit responses and retry with bounded backoff.
- Controllers MUST treat missing responses, stale process sequences, bad checks, and error responses as communication faults for watchdog purposes.

13.6 Process Sequence Enforcement

This section defines normative rules for the application-level `process_sequence` field in `WRITE_ENDPOINT` and `EXCHANGE_ENDPOINTS` payloads. These rules apply in addition to frame-level `sequence` / `ack_sequence` handling (§13.5) and independent of Category A/B stale-frame age checks (§13.4.1 and §13.4.2).

13.6.1 Scope

- `process_sequence` is carried in `LeapEndpointDataHeader` (§13.2) and `LeapExchangeHeader` (§13.3).
- Devices MUST enforce process sequence only after the frame passes transport, owner-session, state, profile, endpoint, and length validation.
- A rejected process sequence MUST NOT apply outputs, MUST NOT advance `highest_process_sequence`, and MUST NOT refresh the owner lease or process watchdog.
- An accepted process sequence MAY refresh the owner lease and process watchdog when the device is in `OP` and the sender is the session owner (§10.2).

13.6.2 Per-session sequence state

For each active owner session, the device MUST maintain:

State	Description
<code>highest_process_sequence</code>	Highest <code>process_sequence</code> value accepted for this owner session
<code>sequence_active</code>	Whether at least one sequence-valid frame has been accepted since the last reset

The device MUST reset sequence state when:

- A new owner lease is granted (`OPEN_SESSION` with owner reply).
- The owner lease ends (`OWNER_RELEASE`, lease expiry, watchdog expiry, or ownership violation).
- The active profile changes (`SELECT_PROFILE` to a different profile ID).

After reset, the first sequence-valid frame establishes the baseline:

`highest_process_sequence` is set to that frame's `process_sequence` and `sequence_active` becomes true.

13.6.3 Acceptance decision

Let `S` be the incoming `process_sequence` and `H` be `highest_process_sequence` after any prior accepts in the current owner session.

Condition	Device action	Typical status code
First accept after reset (<code>sequence_active == false</code>)	Accept; set <code>H = S</code>	—
<code>S < H</code> (regressive)	Reject; no output update	<code>OUT_OF_ORDER (0x000E)</code>
<code>S == H</code> (duplicate cycle)	Reject; no output update	<code>DUPLICATE_SEQUENCE (0x000F)</code>
<code>S == H + 1</code> (next cycle)	Accept; set <code>H = S</code>	—
<code>S > H + 1</code> (gap)	See §13.6.4	— or <code>OUT_OF_ORDER</code>

Devices MUST NOT apply outputs for any rejected row in this table.

When a device returns an explicit error for a rejected cyclic frame, the `LeapErrorPayload.rejected_sequence` field SHOULD contain the failing `process_sequence` value.

13.6.4 Skipped sequence values (gaps)

A gap occurs when `S > H + 1` after at least one prior accept.

Default v1 behavior:

- The device MUST accept the frame and set `H = S` unless the active profile or the frame's process-data flags prohibit gap acceptance.
- The device SHOULD count the gap (`S - H - 1` missed cycle indices) for diagnostics.
- Gap acceptance MUST NOT by itself force a state transition; missed-cycle and watchdog rules in §13.4.1 still apply based on inter-arrival timing.

If the `ALLOW_SKIP` flag (`0x0008`, §13.1) is clear in the endpoint or exchange flags and the active profile does not advertise gap tolerance, the device MAY reject gap frames with `OUT_OF_ORDER` instead of accepting them. Profiles that require strict consecutive sequencing MUST document that policy in the profile descriptor or directory capabilities.

Controllers SHOULD monotonically increment `process_sequence` by one per cyclic cycle under normal operation. Controllers MUST NOT reuse a previously accepted `process_sequence` value while the same owner session remains active.

13.6.5 Exchange replies

For `EXCHANGE_ENDPOINTS`, sequence enforcement is evaluated on the request before outputs are applied or inputs are sampled.

`EXCHANGE_REPLY` payloads SHOULD populate `LeapExchangeStatus` with:

- `latest_process_sequence_consumed` — the `process_sequence` from the accepted request (or the rejected value when returning an error).

- `device_process_sequence` — the device-side sequence counter after processing (MAY equal the consumed value in `v1`).
- `status_code` — `OK` when the exchange was accepted; otherwise the rejection reason from §13.6.3.

13.6.6 Diagnostics

Devices SHOULD increment standard counters (§14.1) on process-sequence events:

Event	Counter
Accepted cyclic frame	<code>PROCESS_CYCLES_ACCEPTED</code>
Duplicate <code>process_sequence</code>	<code>DUPLICATE_SEQUENCES</code>
Regressive or rejected ordering	<code>OUT_OF_ORDER_FRAMES</code>
Age-based stale cyclic frame (Category B)	<code>STALE_PROCESS_FRAMES</code>
Inter-arrival miss (Category A)	<code>PROCESS_CYCLES_MISSED</code>

Devices SHOULD emit `STALE_FRAME_REJECTED` (§14.2) when an age-based stale frame is dropped. Process-sequence rejections SHOULD be visible through counters and MAY be logged as vendor events until a dedicated process-sequence event ID is assigned.

13.6.7 Controller obligations

- Controllers MUST increment `process_sequence` for each new cyclic command intended to apply outputs or sample inputs.
- Controllers MUST treat sustained `DUPLICATE_SEQUENCE`, `OUT_OF_ORDER`, or missing replies as communication degradation and MUST continue lease refresh only while policy allows remaining in `OP`.
- After regaining a valid device following a gap or fault, controllers SHOULD resume with a fresh `process_sequence` only after the management bootstrap sequence completes (§10, §12); reusing pre-fault sequence values across owner changes is forbidden.

14. Diagnostics Service

`LEAP-DIAG` uses service ID `0x0020`. It provides runtime evidence for tuning, fault isolation, commissioning, and conformance testing.

Message Type	Name	Purpose
<code>0x0001</code>	<code>READ_COUNTERS</code>	Read protocol counters
<code>0x0002</code>	<code>COUNTERS_REPLY</code>	Counter payload
<code>0x0003</code>	<code>READ_TIMING</code>	Read loop and transport timing

Message Type	Name	Purpose
0x0004	TIMING_REPLY	Timing payload
0x0005	READ_EVENTS	Read event log entries
0x0006	EVENTS_REPLY	Event log payload
0x0007	TRACE_MARK	Insert a timestamped trace marker

14.1 Standard Counters

Counter ID	Name
0x0001	RX_FRAMES_ACCEPTED
0x0002	RX_FRAMES_REJECTED
0x0003	TX_FRAMES_ACCEPTED
0x0004	TX_FRAMES_DROPPED
0x0005	CRC_FAILURES
0x0006	BAD_LENGTH_FAILURES
0x0007	UNSUPPORTED_MESSAGES
0x0008	DUPLICATE_SEQUENCES
0x0009	LEASE_EXPIRATIONS
0x000A	STATE_TRANSITION_REJECTS
0x000B	PROCESS_CYCLES_ACCEPTED
0x000C	PROCESS_CYCLES_MISSED
0x0010	STALE_PROCESS_FRAMES
0x0011	LATE_PROCESS_FRAMES
0x0012	OUT_OF_ORDER_FRAMES
0x0013	REPLY_TIMEOUTS
0x0014	MAX_REPLY_LATENCY_US
0x0015	LAST_REPLY_LATENCY_US
0x0016	SWITCH_CONGESTION_HINTS
0x8000..	Vendor counters

14.2 Standard Events

Event ID	Name
0x0001	BOOT
0x0002	SESSION_OPENED
0x0003	SESSION_CLOSED
0x0004	OWNER_ACQUIRED
0x0005	OWNER_RELEASED
0x0006	STATE_CHANGED
0x0007	FAULT_ENTERED
0x0008	FAULT_RESET
0x0009	LOCATE_STARTED
0x000A	LOCATE_STOPPED
0x000B	STALE_FRAME_REJECTED
0x000C	WATCHDOG_EXPIRED
0x8000..	Vendor events

14.3 Timing Reply

`LeapTimingReply` reports:

- last, maximum, and minimum cycle time in microseconds
- last and maximum reply latency in microseconds
- remaining process watchdog time in microseconds
- remaining owner lease time in microseconds

15. Watchdog and Safe Output Rules

Each LEAP profile MUST define:

- power-up output defaults
- safe output values
- fault output values
- owner lease timeout
- process-data timeout
- maximum accepted process frame age
- whether inputs remain readable in `SAFE` and `FAULT`

- whether output modes are retained or reset on safe/fault entry

Outputs MUST NOT be applied unless:

- the controller owns the active lease
- the device is in `OP`
- the endpoint direction is controller-to-device
- the endpoint ID and length match the active profile
- the request profile ID matches the active profile
- the latest frame passed all integrity checks
- the frame is not stale, duplicate, or out of order
- the process watchdog is not expired

On owner lease timeout, process-data timeout, malformed cyclic frame, stale frame, or communication loss, the device MUST stop applying stale outputs and transition to `SAFE` or `FAULT` according to the active profile's policy.

16. Fragmentation

Fragmentation is optional. It MUST NOT be used for cyclic process data. It is provided only for infrequent large metadata exchanges — such as directory dumps, object reads, or vendor configuration blobs — that exceed a single Ethernet frame. Devices SHOULD NOT advertise or use fragmented transfers in normal operation.

When the `FRAGMENTED` flag is set, the payload begins with `LeapFragmentHeader`.

Offset	Size	Field	Description
0	4	<code>fragment_group_id</code>	Reassembly key
4	2	<code>fragment_index</code>	Zero-based fragment index
6	2	<code>fragment_count</code>	Total fragments
8	4	<code>total_length</code>	Complete unfragmented payload bytes
12	4	<code>total_crc32c</code>	CRC-32C over complete payload

CRC coverage for fragmented frames:

Two independent CRC fields protect fragmented transfers. Receivers MUST validate both in sequence and MUST NOT act on the payload until both pass.

- `LeapHeader.payload_crc32c` covers the fragment payload bytes carried by the current individual Ethernet frame — the bytes following `LeapFragmentHeader`. This allows each fragment to be validated before it is copied into the reassembly buffer, preventing buffer pollution from corrupt wire data.
- `LeapFragmentHeader.total_crc32c` covers the complete reassembled payload

after all fragments have been received and concatenated. This is the integrity gate that must pass before any application-visible action is taken.

LEAP-PD fragmentation prohibition:

Frames with `service_id == LEAP_SERVICE_PD` MUST NOT have the `FRAGMENTED` flag set and MUST NOT contain a `LeapFragmentHeader`. If a receiver encounters a fragmented process-data frame it MUST immediately discard the frame, MUST NOT update any process sequence state, and MUST increment `RX_FRAMES_REJECTED`. Process-data handlers MUST be free of all reassembly logic; the cyclic I/O path requires deterministic, bounded execution time and must not be blocked or delayed by a concurrent background reassembly operation.

16.1 Reassembly Rules

Receivers MUST enforce all of the following rules. Failure to do so creates exposure to memory exhaustion, state corruption, and injection attacks from malformed or adversarial fragment streams.

Group identification:

- `fragment_group_id` is a per-sender reassembly key scoped to a single session. It is not globally unique across devices or across device reboots.
- `fragment_count` MUST be identical in every fragment of the same group. If a receiver observes any fragment whose `fragment_count` differs from the value in the first fragment received for that `fragment_group_id`, it MUST discard the entire group and respond with `BAD_LENGTH`.

Ordering, duplicates, and index bounds:

- Fragments MUST be accepted regardless of arrival order within a group.
- `fragment_index` is zero-based. Fragments with `fragment_index >= fragment_count` MUST be discarded immediately.
- Duplicate fragments (same `fragment_group_id` and same `fragment_index`) MUST be silently discarded; only the first received copy is retained.
- Because `fragment_index` addresses a distinct logical slot, overlapping content cannot arise from valid senders. Receivers MUST NOT attempt to merge or reconcile content for the same index slot.

Memory and resource limits:

- Receivers MUST limit concurrent active reassembly groups to at most 4 per session. Groups that would exceed this limit MUST be silently dropped.
- Receivers MUST reject any fragment group whose `fragment_count` exceeds 255.
- Receivers MUST reject any fragment group whose `total_length` exceeds `LEAP_MAX_REASSEMBLY_SIZE` (4096 bytes by default). Implementations MUST allocate reassembly buffers as static arrays bounded by this constant; dynamic heap allocation for reassembly is not required and SHOULD be avoided on resource-constrained devices. Groups exceeding the limit MUST be rejected with `BAD_LENGTH`.

- Receivers MUST discard incomplete fragment groups when their reassembly timer expires. The timer MUST be started on receipt of the first fragment of any index in a new group. The normative minimum reassembly timeout is 5 seconds.

CRC gate and state-change prohibition:

- Receivers MUST NOT apply any state change, output write, object write, profile selection, or any other application-visible side effect derived from a fragmented message until all of the following conditions are met:
 1. All `fragment_count` fragments have been received.
 2. The concatenated payload byte length exactly equals `total_length`.
 3. The CRC-32C of the reassembled payload matches `total_crc32c`.
- If the final integrity check fails, the entire group MUST be discarded and the receiver MUST respond with `BAD_CHECK`. Partial or speculative state changes from incomplete or unverified reassembly are not permitted under any condition.

17. Security Model

LEAP v1.0 draft security assumes an isolated machine network. This assumption is not sufficient for routed, shared, or hostile networks.

Initial security requirements:

- Devices ignore output writes from non-owner sessions.
- Owner leases limit stale controller authority.
- Discovery and locate-device requests have no output side effects.
- Malformed frames are rejected without assertions, crashes, or state changes.
- Diagnostics expose owner identity, timeout counters, stale-frame counters, and relevant fault evidence.

Future security extensions SHOULD consider authenticated sessions, frame message authentication codes, role-based write access, secure provisioning, and replay protection across device restarts.

`AUTH_REQUIRED` and `PERMISSION_DENIED` are reserved standard status codes for future authentication and authorization extensions.

For v1 lab deployments, an unauthenticated controller on the isolated cell LAN can claim ownership after the active owner lease expires. That behavior is acceptable only for controlled lab and private machine-cell networks. Routed, shared, plant-wide, or hostile networks require an authentication and authorization extension before LEAP output control is considered production ready.

Deployments MUST NOT use LEAP v1.0 on non-isolated, plant-wide, enterprise IT/OT, or internet-routed networks without an authentication and authorization extension that covers at minimum session establishment, frame integrity, and replay protection. The owner lease mechanism is not a substitute for access control on networks where unauthenticated nodes can join. Using LEAP v1.0 output

control on such networks without an authentication extension is a deployment error regardless of whether physical access to the network is believed to be restricted.

18. Profile Specification Requirements

Device, I/O, motion, sensor, and actuator behavior **MUST** be defined in separate profile specifications rather than embedded in the base protocol. A profile specification defines the process-data contract that a controller and device use after `SELECT_PROFILE` succeeds.

Each profile specification **MUST** define:

- profile ID and profile name
- profile revision policy
- supported endpoint descriptors
- endpoint payload layouts and byte order
- command, status, input, output, safe-output, and fault-output semantics where applicable
- process-data cycle expectations
- maximum accepted frame age
- owner lease and process watchdog defaults
- safe-state and fault-state behavior
- profile-local status and fault codes
- directory objects or TLVs used to describe optional capabilities
- compatibility rules for future profile revisions

A profile specification **MUST NOT** depend on a specific controller application, board layout, connector name, expansion bus, vendor product, or firmware project unless it is explicitly declared as a vendor/project profile in the vendor range. Standard profiles should describe generic capability classes such as digital I/O, analog I/O, motion axes, sensors, or actuators.

The base protocol reserves standard endpoint IDs for common command/status, digital I/O, analog I/O, safe-output, and fault-output use. Profiles **MAY** define additional endpoints when the standard endpoints are insufficient. Vendor or project-specific endpoints **MUST** use the vendor endpoint range.

`DIGITAL_IO_16X16` **SHOULD** be the first golden reference profile for v1.0 interoperability work. It is large enough to exercise byte ordering, endpoint descriptors, process-data exchange, safe outputs, and diagnostics while remaining simple enough for independent implementations and conformance vectors.

18.1 Implementation-Independent Profile Manifest

Devices SHOULD expose profile metadata in a machine-readable manifest format so tools can discover channel mappings without hardcoded assumptions. JSON is the recommended baseline exchange format for v1.0 tooling.

Reference schema file: `leap-manifest-schema.json`.

Minimum manifest fields:

- `vendor_id` (16-bit integer, aligned with `LeapIdentity.vendor_id`)
- `product_code` (integer)
- `product_name` (string)
- `revision` (integer)
- `supported_services` (array of 16-bit service IDs)
- `profiles` (array of profile objects)

`serial_number` is intentionally not required. The manifest describes a device model or firmware build, not an individual unit. Serial number is instance data, read at runtime from `LeapIdentity` via `LEAP-DISC` or `LEAP-DIR`. Include it in the manifest when generating a per-unit file, but tooling MUST NOT require it to be present for channel mapping or profile discovery purposes.

Each profile object SHOULD include:

- `profile_id`
- `profile_name`
- `process_data.input_size_bytes`
- `process_data.output_size_bytes`
- `process_data.channels[]` entries with:
 - `name`
 - `type` (`BIT`, `UINT8`, `INT8`, `UINT16`, `INT16`, `UINT32`, `RAW`)
 - `bit_offset`
 - `direction` (`INPUT`, `OUTPUT`, `DIAGNOSTIC`)
 - `size_bytes` or `bit_width` when `type = RAW`

Manifest content MUST be consistent with `LEAP-DIR` objects and endpoint descriptors. If manifest and live directory data conflict, controllers MUST trust live `LEAP-DIR` responses.

19. Conformance Targets

19.1 Formal Conformance Claim

A device implementation may claim **LEAP v1.0 conformant device** status only if it:

- supports `LEAP-DISC`, `LEAP-MGMT`, and `LEAP-DIR`
- supports `LOCATE_DEVICE` discovery commissioning behavior
- implements the required state machine and owner-lease safety rules
- supports at least one profile from the standard or vendor profile ranges
- passes published golden vectors and CRC validation vectors

A controller implementation may claim **LEAP v1.0 conformant controller** status only if it:

- performs discovery, session ownership, and state transition control
- performs cyclic `LEAP-PD` exchange with sequence/timeout handling
- validates directory/profile metadata and reads diagnostics on failures

A minimally conformant LEAP v1.0 device supports:

- common header validation
- CRC and length rejection without side effects
- discovery `HELLO`, `IDENTIFY`, and `LOCATE_DEVICE`
- mandatory identity object
- management `OPEN_SESSION`, `HEARTBEAT`, `SET_STATE`, `CLOSE_SESSION`, `OWNER_RELEASE`, and `FAULT_RESET`
- directory read of identity, supported services, active profile, and endpoints
- process-data `EXCHANGE_ENDPOINTS`
- switch-safe stale-frame and sequence checks
- diagnostics counters and timing
- owner lease timeout to safe outputs
- deterministic rejection of malformed frames

A minimally conformant LEAP v1.0 controller supports:

- raw Ethernet discovery
- MAC-address device discovery and identity parsing
- locate-device commissioning request
- session ownership
- state transition to `OP`
- cyclic exchange with sequence, timestamp, profile, and timeout tracking
- device directory and endpoint descriptor parsing

- diagnostics readback on error
- safe recovery from cable pull, missing replies, and controller shutdown

20. Validation Plan

Unit-level validation:

- Header parser accepts valid frames and rejects malformed lengths.
- Transmit path pads small frames to the minimum Ethernet payload requirement without changing `payload_length` or payload CRC coverage.
- Header and payload CRC implementations match published check values and golden frame vectors.
- CRC failures are rejected without state changes.
- Unsupported service and message IDs return deterministic errors.
- TLV parser skips unknown valid TLVs and rejects invalid lengths.
- Owner lease state machine rejects non-owner output writes.
- Process watchdog transitions from `OP` to `SAFE`.
- Stale process frames are rejected and counted.
- One-outstanding cyclic exchange per device/session owner is enforced.
- `REBOOT_RECOVERY` breaks only same-MAC stale owner sessions and returns the device to `CONFIGURED`.
- `EXCHANGE_ENDPOINTS` writes status at offset `36 + write_length`.

Device validation:

- Discovery returns identity and profile data.
- Locate-device activates only human-facing indicators.
- Session open grants owner lease.
- State path reaches `OP`.
- Cyclic exchange updates application sequence acknowledgement.
- Output control and input readback work for profile endpoints.
- Stopped cyclic exchange causes safe transition.
- Diagnostics counters increment on expected faults.

Controller validation:

- Adapter selection and raw transmit/receive work on the target controller platform.
- Discovery finds one device on an isolated network interface.
- Directory parser builds the endpoint table.
- Cyclic exchange meets target loop period.
- Controller diagnostics report device, controller, and reply timing.
- Diagnostics counters increment on stale frames and expected faults.
- Non-owner write attempts are rejected and visible.

20.1 Tooling Artifacts

The following files are part of the v1.0 interoperability package:

- `leap_dissector.lua`: initial Wireshark Lua dissector for LEAP headers and common process-data payload decoding (not a complete service dissector).
- `LEAP_GOLDEN_FRAME_VECTORS.md`: canonical CRC and frame vectors for automated parser/CRC verification.
- `leap-manifest-schema.json`: implementation-independent JSON schema for device and profile manifests.

21. Versioning and Extension Rules

- Major version changes may break wire compatibility.
- Minor version changes MUST preserve the common header and existing service semantics.
- Minor-version negotiation rule: if both peers advertise compatible major version `1`, they MUST operate using the lower minor version supported by both peers (minimum of controller and device minor versions).
- Unknown standard service IDs below `0x8000` are rejected.
- Unknown vendor service IDs MAY be rejected when no matching extension is enabled.
- Unknown TLVs with valid lengths are skipped.
- Reserved bits are sent as `0` and ignored on receive unless later assigned.
- Profile IDs are immutable once published.
- New incompatible field layouts require a new profile ID.
- Vendor services, TLVs, profiles, counters, events, and objects MUST stay in their assigned extension ranges.

22. Open Review Items

The v1.0 maturity gate is intentionally small. Before LEAP is called v1.0-ready, the specification and release package MUST include golden packet capture examples that match the fixed CRC/session rules in this document. This is the minimum needed for an independent implementation without out-of-band clarification.

- Choose and register the production EtherType.
- Keep `0x88B6` as default development EtherType and `0x88B5` as configurable experimental alternate; do not ship products with experimental EtherTypes as immutable defaults.
- If LEAP is deployed on shared or enterprise IT/OT infrastructure, pursue a formally assigned production EtherType before broad rollout.
- Validate stale-frame thresholds and counter behavior for Category A and Category B devices under real switch congestion and broadcast-storm conditions.

- Confirm the conservative 50 byte minimum transmitted Ethernet payload rule across target raw-socket and embedded Ethernet drivers.
- Version and publish `leap-manifest-schema.json` with semantic schema revision metadata.
- Decide when authentication moves from future extension to required feature.
- Validate `LEAP_GOLDEN_FRAME_VECTORS.md` vectors against independent implementations and packet-capture outputs.
- Verify profile IDs used in golden vectors exactly match the profile registry and manifest declarations.
- Publish golden packet captures for `DIGITAL_IO_16X16`.
- Validate `leap_dissector.lua` against real captures and profile variants.

22.1 Pre-Flight Tagging Checklist (v1.0.0)

Before tagging the v1.0.0 release commit, the following checklist SHOULD be completed and archived with test artifacts:

- ☐ **EtherType validation:** confirm development tests run on `0x88B6`; verify production EtherType registration plan for shared enterprise IT/OT deployment.
- ☐ **Structure sanity checks:** compile with target GCC/Clang/MSVC toolchains and confirm all `LEAP_STATIC_ASSERT` layout checks pass.
- ☐ **Golden vector compliance:** run automated parser/CRC tests against `LEAP_GOLDEN_FRAME_VECTORS.md` and confirm expected values (`0x31c3`, `0xE3069283`, and frame-level vectors).
- ☐ **Padding/memory insulation:** validate transmit/receive handling of the 50 byte minimum Ethernet payload rule and verify no data leakage from padding into application-visible payload processing.
- ☐ **Safety watchdog under stress:** execute switch congestion and broadcast-storm tests to confirm Category A and Category B stale-frame logic transitions safely to `SAFE / FAULT` when thresholds are exceeded.
- ☐ **Profile mapping consistency:** verify profile IDs and field layouts match across `leap_protocol.h`, `LEAP_GOLDEN_FRAME_VECTORS.md`, and `leap-manifest-schema.json`.
- ☐ **Zero-payload CRC boundary:** verify that `header_crc16` is computed correctly when `payload_length == 0` and that the transmit path pads the remaining Ethernet floor bytes with explicit zeros without leaking adjacent stack or heap content onto the wire.
- ☐ **Malformed-frame watchdog isolation:** confirm that frames rejected at header CRC, payload CRC, length, magic, or version checks do not reset, reload, or extend any active owner lease or process watchdog timer. Invalid frames must be silently discarded at the protocol boundary; the watchdog must advance toward expiry as if no frame had arrived.